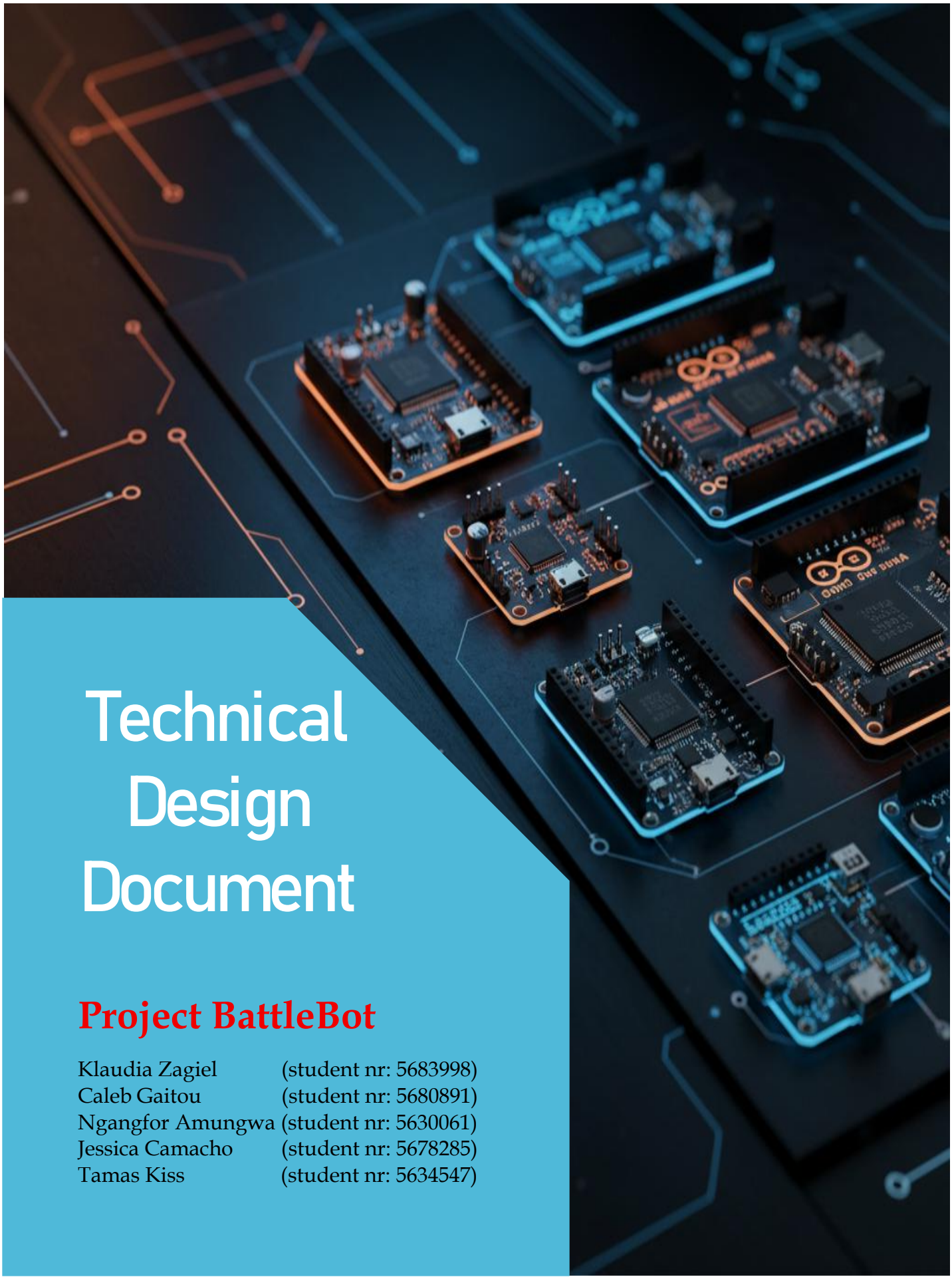


Technical Design Document

Project BattleBot

Klaudia Zagiel (student nr: 5683998)
Caleb Gaitou (student nr: 5680891)
Ngangfor Amungwa (student nr: 5630061)
Jessica Camacho (student nr: 5678285)
Tamas Kiss (student nr: 5634547)

Contents

Introduction 3
- **Purpose of This Document** 3
- **Scope** 3

Overall System Architecture 4
- **System Overview** 4
- **Architecture Diagrams** 5

Hardware Design 6
- **Hardware Components** 6
- **Pin Configuration** 7

Software Design 9
- **Software Structure** 9
- **Program Flow** 9
- **Control Logic Approach** 10

Subsystem Technical Design 11
- **Straight-Line Subsystem** 11
- **Circular-Line Subsystem** 11
- **Maze Subsystem** 12

Distance and Movement Calculations 13
- **Wheel-Based Distance Measurement** 13

Testing and Calibration 14
- **Subsystem Testing and Calibration** 14
- **Integration Testing** 14

Limitations and Risks 15
- **Hardware Limitations** 15
- **Environmental Factors** 16

Maintainability and Extensibility 17
- **Maintainability** 17
- **Extensibility** 18

Conclusion 19

Introduction

Purpose of This Document

This document describes the **technical implementation** of the BattleBot Relay Race System.

It explains how the functional requirements described in the Functional Design are realized using hardware and software.

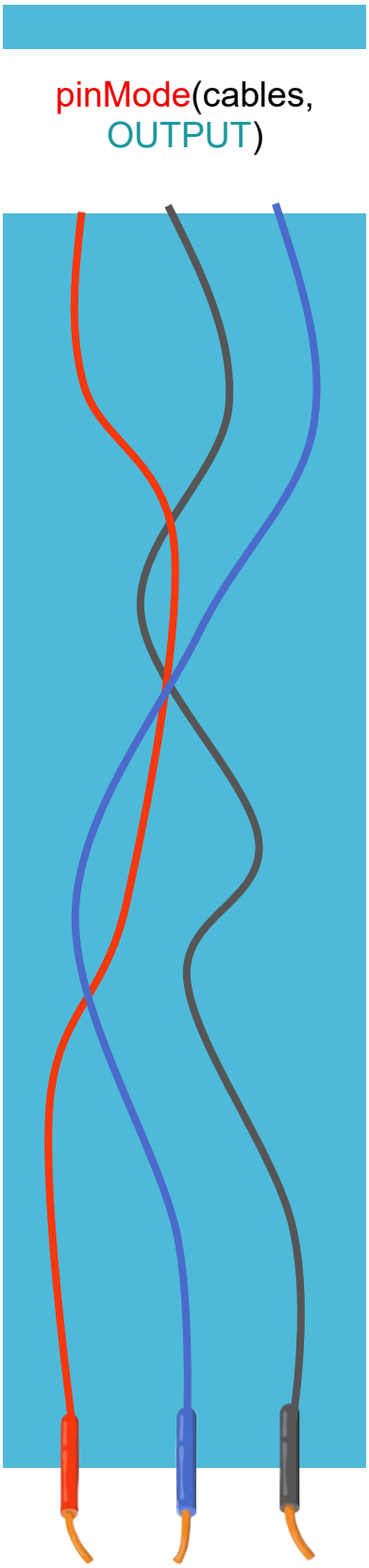
Given the fact that is a group project, this document will include **separate technical sections** for each relay subsystem.

Scope

This document covers:

- Overall system architecture
- Hardware design choices
- Software structure
- Subsystem-specific technical implementations

Detailed source code is maintained separately.



Overall System Architecture

System Overview

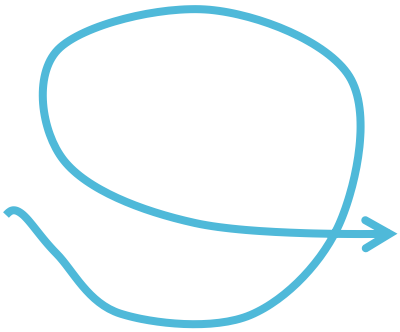
The relay system consists of **three autonomous robotic subsystems**, each controlled by an Arduino Nano.

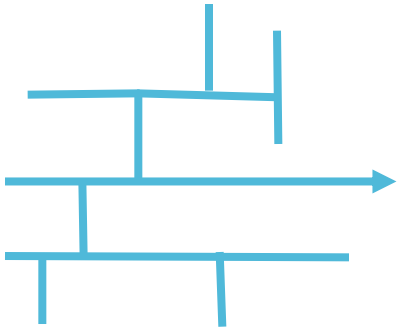
The subsystems operate sequentially and interact through a **mechanical relay handover** using a shared pillar.

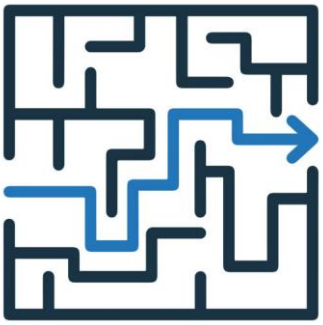
Subsystems:

1. Circular-Line Robot
2. Straight-Line Maze Robot
3. Maze Robot

There is no electronic communication between subsystems and coordination is achieved through mechanical positioning and timing.

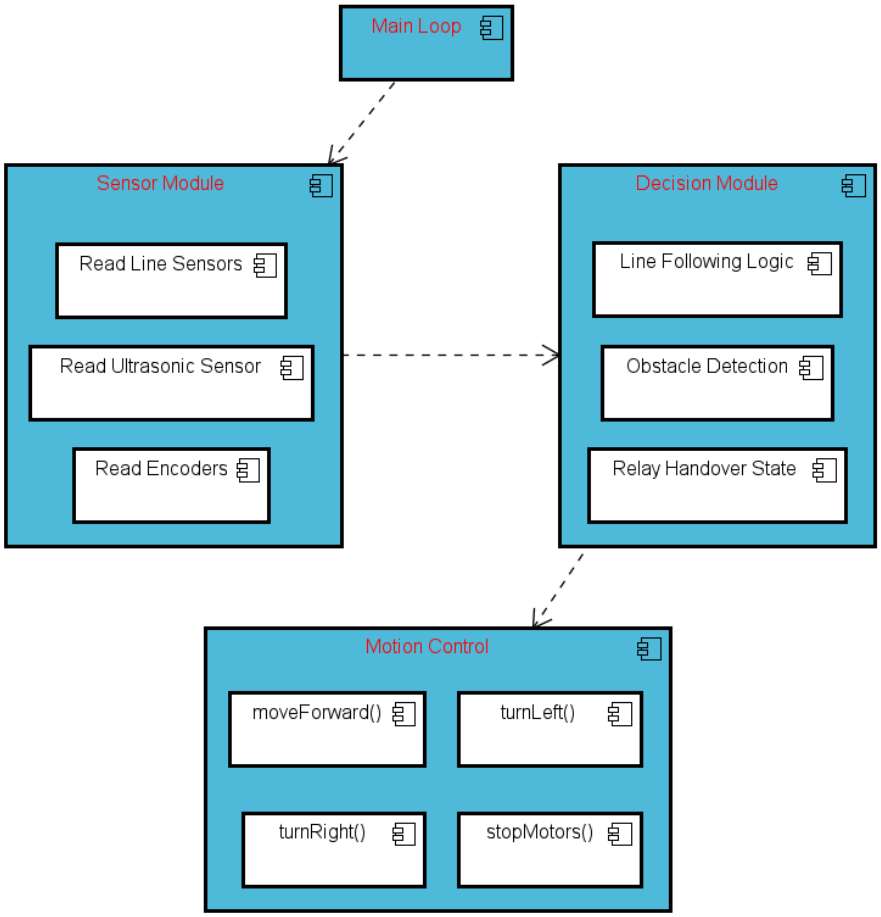




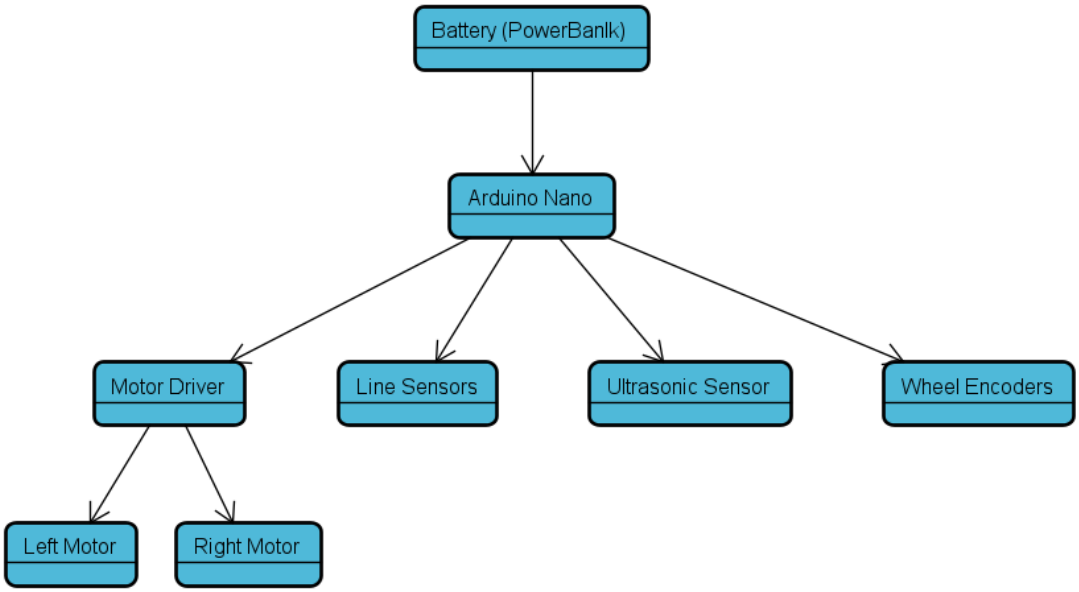


Architecture Diagrams

1. Software Architecture



2. Internal Architecture of One Robot



Hardware Design

Hardware Components

All subsystems share the same hardware structure:

Component	Description
Arduino Nano	Main microcontroller responsible for executing control logic and coordinating sensors and actuators.
DC motors (x2)	Provide differential drive movement for forward motion and turning.
Motor driver (H-bridge)	Controls motor direction and speed by switching voltage polarity.
Line sensors (x8 analog)	Used to detect black track lines and guide navigation.
Wheel encoders (x2)	Provide 20 pulses per wheel rotation to measure distance and turning angles.
Battery (PowerBank)	Primary power supply for autonomous operation.
Ultrasonic Sensor (HC-SR04)	Measures distance to obstacles using echo-based time-of-flight principle.
Gripper Mechanism	Mechanical actuator used to hold and release the relay pillar.
Bluetooth Module	Enables optional wireless communication (optional implementation).
I2C Expansion PCB	Expands input/output capability and simplifies sensor connections via I2C communication.
LEDs	Provide visual feedback for debugging or status indication.
NeoPixel LEDs	Programmable RGB LEDs used for advanced visual feedback (optional implementation).
Push Buttons	Used for manual input, testing routines, or mode selection during development.
Jumper Cables	Electrical wiring used to connect sensors and modules to the Arduino.
USB Programming Cable	Used to upload code from computer to Arduino Nano.
USB Power Cable	Supplies power during development and testing.

Pin Configuration

1. **Straight-line subsystem robot**

Subteam: Caleb Gaitou & Ngangfor Amungwa

Function	Arduino Pin
Left Motor Forward	10
Left Motor Backward	11
Right Motor Forward	9
Right Motor Backward	4
Ultrasonic Trigger	12
Ultrasonic Echo	5
Left Encoder (rotation)	3
Right Encoder (rotation)	2
Line Sensor Most Left	A7
Line Sensor Center Left	A6
Line Sensor Right Left	A5
Line Sensor Left Center	A4
Line Sensor Right Center	A3
Line Sensor Left Right	A2
Line Sensor Center Right	A1
Line Sensor Most Right	A0
Gripper Servo	12
LED	7

2. **Circular-line subsystem robot**

Subteam: Klaudia Zagiel & Jessica Camacho

Function	Arduino Pin
Left Motor Forward	10
Left Motor Backward	11
Right Motor Forward	9
Right Motor Backward	6
Ultrasonic Trigger	4
Ultrasonic Echo	13
Left Encoder (rotation)	2
Right Encoder (rotation)	3
Start Button	7
Line Sensor Most Left	A7
Line Sensor Center Left	A6
Line Sensor Right Left	A5
Line Sensor Left Center	A4
Line Sensor Right Center	A3
Line Sensor Left Right	A2
Line Sensor Center Right	A1
Line Sensor Most Right	A0
Gripper Servo	12

LED	8

3. **Maze subsystem robot**

Subteam: Tamas Kiss

Function	Arduino Pin
Left Motor Forward	5
Left Motor Backward	6
Right Motor Forward	7
Right Motor Backward	8
Ultrasonic Trigger	9
Ultrasonic Echo	10
Left Encoder (rotation)	2
Right Encoder (rotation)	3
Button	4
Gripper Servo	12
LED	13

Software Design

Software Structure

The software is written in **Arduino C++** and follows a **modular structure**, because instead of creating everything inside the loop() we will be creating functions to do each "task".

Main characteristics:

- **Functional decomposition –** Breaking a big task into small tasks, so we can create the functions following a step by step structure;
- **Reusable movement functions –** saving us from repeating the same pieces of code multiple times (memory saver);
- **Sensor abstraction –** clean architecture that divides sensor logic from decision logic;
- **State-based logic –** we define robot states, making it behave predictably;

Program Flow

Each subsystem follows a similar execution flow:

1. Calibration variables at the very top for easy access and tunning (per student)
2. Initialization (setup)
    - Configure pins
    - Initialize serial (for debugging)
    - Attach interrupts
3. Main control loop (loop)
    - Read sensors
    - Decide what to do
    - Execute movement
4. State-based behavior execution with functions at the bottom

Control Logic Approach

The robots use a **state-machine–like structure**, where behavior depends on sensor input and current state.

Example states used:

- Idle
- grabObject
- moveInAStraightLine
- stopAtTranfer
- realseObject
- followLine
- avoidWalls

Subsystem Technical Design

Straight-Line Subsystem

*(**Subteam:** Caleb Gaitou & Ngangfor Amungwa)*

1. **Responsibilities:**
   - Pick pillar from pre-assigned zone when the circular line subsystem drops it
   - Figure out the maze by following the line
   - Speed stabilization
   - Detect object and avoid it
   - Precise stopping at handover point
   - Release pillar

2. **Subsystem Functions:**
   - void grabObject();
   - void followStraightLine();
   - void detectCircularLineTrackStart();
   - void dropObject();

3. **Movement Control Functions:**
   - void moveForward();
   - void turnRight90();
   - void turnLeft90();

Circular-Line Subsystem

*(**Subteam:** Klaudia Zagiel & Jessica Camacho)*

1. **Responsibilities:**
   - Start race at race flag lift
   - Pick pillar from handover zone
   - Follow circular black track
   - Maintain orientation
   - Avoid object when one shows in its path
   - Detect drop off zone
   - Release pillar
   - Back a little to create safe space

2. **Subsystem Functions:**
   - void grabObject
   - void followCircularLine();
   - void detectMazeEntrance();
   - void releaseObject();

3. **Movement Control Functions:**
   - void moveForward();
   - void turnLeft90();
   - void turnRight90();

Maze Subsystem

*(**Subteam:** Tamas Kiss)*

1. **Responsibilities:**
   - Pick up pillar from handover zone
   - Navigate maze
   - Avoid obstacles
   - Reach maze exit
   - Drop pillar

2. **Subsystem Functions:**
   - void grabPillar();
   - void findFinishLine ();
   - void detectMazeEntrance();
   - void releaseObject();
   - void rememberPath();
   - void navigateMaze();

3. **Movement Control Functions:**
   - void moveForward();
   - void moveBackwards();
   - void turnLeft90();
   - void turnRight90();

Distance and Movement Calculations

Wheel-Based Distance Measurement

*(**Made by:** Klaudia Zagiel)*

Wheel diameter: 6.5 cm

Formula: $C = \pi \times d = 3.14 \times 6.5$

Wheel circumference: ≈ 20 cm

Rotation sensor output:

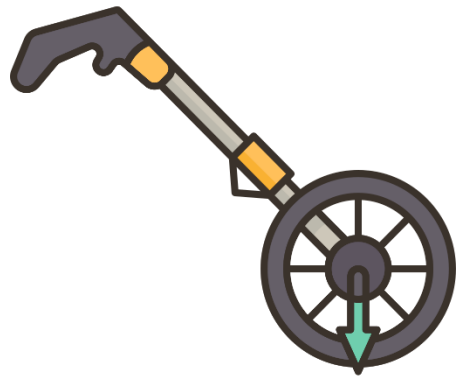
- 20 pulses per full rotation

Therefore:

- 1 pulse ≈ 1 cm
- 100 pulses ≈ 1 meter

This method is used to:

- Control forward movement distance
- Calibrate turning angles
- Improve movement accuracy



Ultrasonic Distance Calculation

*(**Made by:** Tamas Kiss)*

The HC-SR04 sensor measures distance using time-of-flight.

Formula:
Distance = (Duration x Speed of Sound) / 2

Where:

- Speed of sound ≈ 0.034 cm/µs
- Duration = echo return time in microseconds
- Division by 2 accounts for forward and return travel

In implementation:
The distance value is compared against a threshold (for example, 20 cm) to determine obstacle detection.

Testing and Calibration

Subsystem Testing and Calibration

The following calibration procedures were performed:

- Motor speed calibration to ensure straight movement
- Turn calibration to achieve approximately 90° turns
- Line sensor threshold adjustment for reliable track detection
- Encoder pulse verification to validate distance calculations
- Stopping distance calibration at relay handover points

Each subsystem was required to perform its core functionality consistently before proceeding to integration testing.

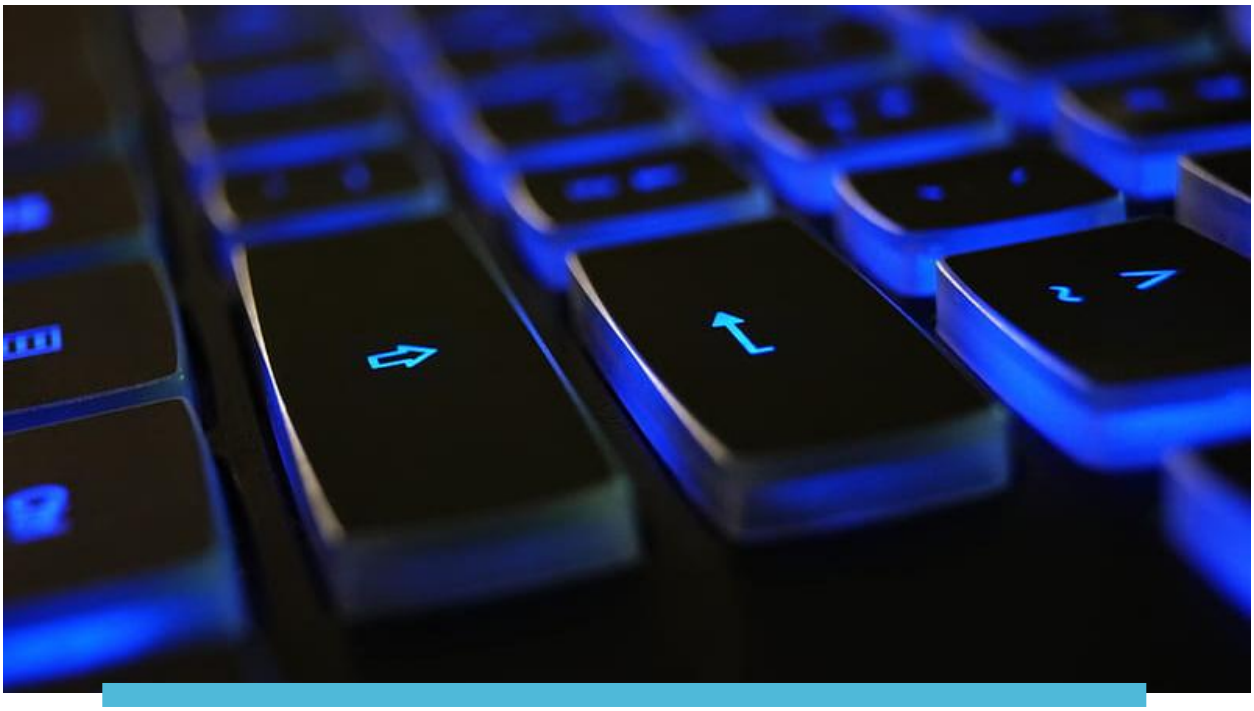
Integration Testing

After individual subsystem validation, full relay testing will be conducted.

Integration testing includes:

- Verification of stopping accuracy at relay transfer points
- Testing of pillar handover stability between robots
- Full relay execution from start to maze exit
- Repeated full-run testing to verify reliability
- Observation and correction of failure scenarios such as misalignment or timing mismatch

Before exam day we all these tests must be done to ensure stability and reliability of the full relay run, **without manual intervention**.



Limitations and Risks

Hardware Limitations

1. **Microcontroller Constraint**

   The system is based on the Arduino Nano, which provides limited SRAM and processing capacity.

   This restricts the complexity of algorithms and limits the use of memory-intensive features such as advanced filtering or large data structures.

   Impact:
   Software must remain efficient and modular. Real-time processing must be lightweight.

2. **Sensor Precision**

   Line sensors and ultrasonic sensors may produce unstable readings due to noise, reflection angles, and surface inconsistencies.

   Impact:
   Incorrect sensor readings may cause temporary deviation from the track or delayed obstacle detection.

Mitigation:
Threshold calibration and repeated measurements are used to improve reliability.

3. **Encoder Accuracy**

Wheel encoders provide 20 pulses per rotation. Small mechanical tolerances, wheel slippage, or uneven surfaces may affect pulse accuracy.

Impact:
Distance calculations and 90° turns may deviate slightly from expected values.

Mitigation:
Calibrations was performed to minimize error.

4. **Mechanical Alignment**

Mechanical tolerances in chassis construction, wheel mounting, and gripper positioning may affect straight-line stability and relay handover precision.

Impact:
Small misalignments may accumulate during long movements.

Environmental Factors

1. **Lighting Conditions**

Line sensors depend on surface contrast between black track lines and the background. Changes in ambient lighting may affect detection reliability.

Impact:
Reduced contrast may cause temporary tracking instability.

2. **Surface Friction Variations**

Different surface materials may influence wheel traction and turning behavior.

Impact:
Movement calibration values may vary between test environments.

3. **Battery Voltage Variation**

Battery discharge over time may affect motor speed and sensor stability.

Impact:
Speed reduction may influence stopping accuracy and timing between subsystems.

Maintainability and Extensibility

Maintainability

The BattleBot relay system was designed with maintainability in mind, both at hardware and software level.

The software is divided into **logical functions**, and **in a modular structure** which ensures individual behaviors, which can be modified or debugged independently without affecting the entire system.

This method improves readability, simplifies debugging, and reduces risk of introducing new errors when modifying functionality.

Besides this, we have implemented clear pin configurations, where **all hardware connections are defined using constant variables** at the beginning of the program.

This allows hardware modifications to be implemented by changing only the pin definitions, without rewriting control logic, which means simplified hardware replacements and reconfigurations.

Lastly we maintained a subsystem separation which allowed **each team to test and improve its subsystems without affecting the others**, in a parallel development that facilitated troubleshooting and future extensions without major redesigns.

Extensibility

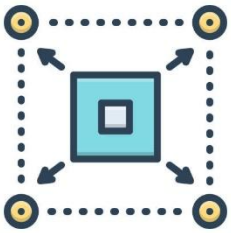
Sensors expansions could be implemented to improve navigation precision, for example, with gyroscope, or infrared sensors.

Communication could be expanded with the included Bluetooth module which allows potential future implementation of wireless monitoring or remote diagnostics.

Algorithm improvements like movement control could be enhanced using, for example:

- PID control line following
- Closed-loop speed regulation
- Improved state machine logic

Which the current modular design makes it possible for such improvements to be implemented incrementally.



Conclusion

This Technical Design document
describes the hardware and
software architecture
of the BattleBot Relay Race System.

The modular approach allows
each subsystem to be
developed independently while
ensuring successful system integration.

NHL STENDEN
hogeschool